

Description

给定含有 n 个元素的多重集合 S ，每个元素在 S 中出现的次数称为该元素的重数。多重集 S 中重数最大的元素称为众数。例如， $S = \{1, 2, 2, 2, 3, 5\}$ 。多重集 S 的众数是2，其重数为3。对于给定的由 n 个自然数组成的多重集 S ，计算 S 的众数及其重数。如果出现多个众数，请输出最小的那个。

Input

输入数据的第1行是多重集 S 中元素个数 n ($n < 1300000$)；接下来的 n 行中，每行有一个最多含有5位数字的自然数，。

Output

输出数据的第1行给出众数，第2行是重数。

Samples

Sample #1

Input	Output
6 1 2 2 2 3 5	2 3

方法一:map映射法

算法分析

遍历多重集合 S ,由于本题考查的是集合的众数和重数，两者有一种天然的一一对应关系，可以使用一个 `map` 维护一个元素与其重数的映射关系。每遍历一个元素 `num`，使 `map[num]++`

遍历完成后，`map` 中保存了每个元素的重数，然后遍历`map`，找到最大的重数，它就是要输出的重数，其对应的 `key` 就是要输出的众数。

`std::map` 基于红黑树，其遍历顺序依据标准库要求按照键值升序遍历，符合题目要求:

如果出现多个众数，请输出最小的那个。

这也是为什么没有使用 `std::unordered_map` 的原因，因为它的遍历是无序的，基于哈希表实现，即使它的插入和查找操作达到了 $O(1)$

伪代码

```
1 input (n)
2 for(i=0->n-1){
3     input (num)
4     map[num]++
5 }
6 int zs=0,cs=0
7 for(key in map){
8     if(map[key]>cs){
9         zs=key
10        cs=map[key]
11    }
12 }
13 print(zs)
14 print(cs)
```

具体实现

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 map<int,int> myMap;
4 int main(){
5     int n,num;
6     cin>>n;
7     for(int i=0;i<n;i++){
8         cin>>num;
9         myMap[num]++;
10    }
11    int zs=0,cs=0;
12    for(auto it=myMap.begin();it!=myMap.end();it++){
13        if(it->second>cs){
14            zs=it->first;
15            cs=it->second;
16        }
17    }
18    cout<<zs<<endl;
19    cout<<cs<<endl;
20    system("pause");
21 }
```

复杂度分析

- 第一层循环:

$$N_1 = 0 + \sum_{i=1}^{n-1} \log i = \log(n-1)!$$

- 第二层循环:

$$N_2 = n \cdot \log n$$

$$N = N_1 + N_2 = n \log n + \log(n-1)!$$

因为当 $n \geq 1$ 时, $N_2 \geq N_1$

proof:

令 $f(n) = \log(n-1)!$

$g(n) = n \log n$

做 e 的指数,不会改变两者的大小关系:

$e^{f(n)}$ 和 $e^{g(n)}$

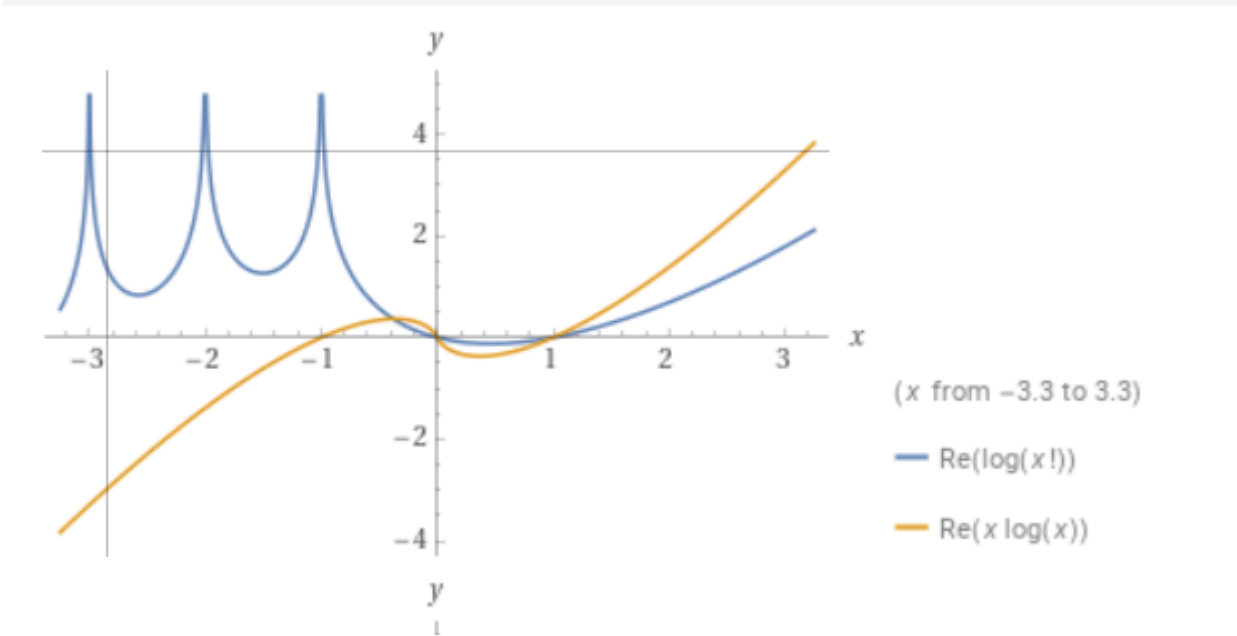
即:

$$e^{\log(n-1)!} = (n-1)!$$

$$e^{n \log n} = n^n$$

当 $n \geq 1$ 时, $(n-1)! \leq n^n$

即: $g(n) \geq f(n)$, $n \geq 1$



所以时间复杂度是 $O(N\log N)$

方法二：排序法

算法分析

可以通过排序来获得元素与其重数的对应关系，因为集合中的元素在排序后，相同的元素，肯定是排列在一起的。然后我们遍历排序后的S数组，使用打擂台的方式得出众数和重数。

伪代码

```
1 input(n)
2 for(i=0->n-1){
3     input(s[i])
4 }
5 sort(s[])
6 int temp_zs=s[0],temp_cs=0,zs=0,cs=0;
7 for(i=0->n-1){
8     if(s[i]==temp_zs){
9         temp_cs++;
10        if(i!=s.size()-1)
11            continue;
12    }
13    else{
14        if(temp_cs>cs){
15            zs=temp_zs
16            cs=temp_cs;
17        }
18        temp_zs=s[i]
19        temp_cs=1
20    }
21 }
22 print(zs)
23 print(cs)
```

具体实现

```
1 #include<bits/stdc++.h>
2 using namespace std;
3
4 vector<int> s;
```

```

5 void swap(vector<int>& a, int i, int j){
6     int temp=a[i];a[i]=a[j];a[j]=temp;
7 }
8 void qsort(int l,int r){
9     if(l>=r) return;
10    int i=l-1,j=r+1,mid=s[l+r>>1];
11    while(i<j){
12        do i++; while(s[i]<mid);
13        do j--; while(s[j]>mid);
14        if(i<j){
15            swap(s,i,j);
16        }
17    }
18    qsort(l,j);
19    qsort(j+1,r);
20 }
21 int main(){
22     int n,num;
23     cin>>n;
24     for(int i=0;i<n;i++){
25         cin>>num;
26         s.push_back(num);
27     }
28     qsort(0,n-1);
29     int temp_zs=s[0],temp_cs=0,zs=0,cs=0;
30     for(int i=0;i<s.size();i++){
31         if(s[i]==temp_zs){
32             temp_cs++;
33             if(i!=s.size()-1)
34                 continue;
35         }
36         if(temp_cs>cs){
37             zs=temp_zs;
38             cs=temp_cs;
39         }
40         temp_zs=s[i];
41         temp_cs=1;
42     }
43     cout<<zs<<endl;
44     cout<<cs<<endl;
45     system("pause");
46 }

```

复杂度分析

- 第一个循环

$$N_1 = O(N)$$

- 排序

$$N_2 = O(N \log N)$$

- 第二个循环

$$N_3 = O(N)$$

综上复杂度是 $O(N \log N)$

方法三: 分治法

算法分析

可以采用分治的思想，对于集合 S 中序列的众数和对应重数，我们可以将从 S 的规模入手，进行子问题的分割，递归的分割，直到集合分为单元素集合时，停止分割，此时每个单元素是该单元素集合的众数，重数是1,然后合并子问题，得到原集合 S 的众数和重数。此方法的难点之一是如何进行子问题的合并，我们可以采用关联容器 `map` 来保存元素和其重数的映射关系，这样保证了分割子问题之间的数据共享。

伪代码

```
1 pair<int,int> findModeAndCount(int s[], int l, int r, myMap){
2     if(l==r){
3         myMap[s[l]]++
4         return {s[l],myMap[s[l]]}
5     }
6     int mid = (l+r)>>1
7     pair<int,int> lRes=findModeAndCount(s,l,mid)
8     pair<int,int> rRes=findModeAndCount(s,mid+1,r)
9
10    if(lRes.second==rRes.second){
11        return lRes.first<rRes.first?lRes:rRes
12    }
13    else{
14        return lRes.second>rRes.second?lRes:rRes
15    }
16 }
17
18 main(){
19     input(n)
20     input(s[])
21     pair<int,int> res = findModeAndCount(0,n-1)
22     print(res.first)
23     print(res.second)
24 }
```

具体实现

```
1 #include<bits/stdc++.h>
2 using namespace std;
3
4 pair<int,int> findModeAndCount(vector<int>& s, int l, int r, unordered_map<int, int>& myMap){
5     if(l==r){
6         myMap[s[l]]++;
7         return {s[l],myMap[s[l]]};
8     }
9     int mid = (l+r)>>1;
10    pair<int,int> lRes = findModeAndCount(s,l,mid,myMap);
11    pair<int,int> rRes = findModeAndCount(s,mid+1,r,myMap);
12    if(lRes.second==rRes.second){
13        return lRes.first<rRes.first?lRes:rRes;
14    }
15    else{
16        return lRes.second>rRes.second?lRes:rRes;
17    }
18 }
19
20 int main(){
21     vector<int> s;
22     unordered_map<int,int> myMap;
23     int n,num;
24     cin>>n;
```

```

25     for(int i=0;i<n;i++){
26         cin>>num;
27         s.push_back(num);
28     }
29     pair<int,int> res = findModeAndCount(s,0,n-1,myMap);
30     cout<<res.first<<endl;
31     cout<<res.second<<endl;
32     system("pause");
33 }

```

复杂度分析

- 输入 S :

$O(N)$

- 函数 `findModeAndCount()`

考虑分治策略，每次都是从集合的中部分割,每次递归内操作皆是常数操作，可以写出时间复杂函数的递推式:

$$\begin{cases} T(n) = 2 \cdot T(n/2) + O(1), & n \geq 2 \\ T(1) = O(1), & n = 1 \end{cases}$$

依据解决关于递归函数时间复杂度的[Master Theorem](#), $T(n) = aT(n/b) + f(n)$

$$a = 2, b = 2, f(n) = O(1)$$

$$n^{\log_b a} = n^{\log_2 2} = 1 = f(n)$$

$$T(n) = O(n^{\log_b a} \log n) = O(\log n)$$

所以时间复杂度: $O(\log N)$

综上时间复杂度是 $O(N)$